Tutorial 06: El Primer Estrangulamiento

Migrando el Servicio de Compras de Django a Flask (v2)

Arquitectura de Software 2026 - Prof. Lucius

1. Introducción

Siguiendo el **Strangler Pattern**, en este laboratorio extraeremos una funcionalidad crítica del monolito (Django) y la moveremos a un microservicio ligero (Flask). El objetivo es que el usuario final siga usando la plataforma sin notar que, al comprar, está saliendo de un sistema y entrando a otro.

2. Paso 1: Creación del Microservicio (Flask)

Cree una nueva carpeta en la raíz de su proyecto llamada `microservicio_pagos/`. Dentro de ella, cree el archivo `app.py`:

```python
from flask import Flask, request, jsonify
import os

app = Flask(__name__)

@app.route('/api/v2/comprar', methods=['POST'])
def realizar_compra():
    data = request.get_json()

    # Simulacion de logica de negocio extraida
    producto_id = data.get('producto_id')
    cantidad = data.get('cantidad', 1)

    if not producto_id:
        return jsonify({"error": "Falta el ID del producto"}), 400

    return jsonify({
        "mensaje": "Compra procesada exitosamente por el Microservicio Flask (v2)",
        "producto_id": producto_id,
        "cantidad": cantidad,
        "status": "Aprobado"
    }), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Listing 1: app.py - El nuevo motor de pagos

3. Paso 2: Dockerizar el "Órgano"Nuevo

Dentro de la carpeta `microservicio_pagos/`, cree un `Dockerfile` específico para Flask:

```
FROM python:3.11-alpine
WORKDIR /app
RUN pip install flask gunicorn
COPY . .
EXPOSE 5000
CMD ["gunicorn", "app:app", "--bind", "0.0.0.0:5000"]
```

Listing 2: Dockerfile para Flask

4. Paso 3: Actualizar la Orquestación (docker-compose.yml)

Modifique su archivo `docker-compose.yml` en la raíz para incluir el nuevo servicio. Observe cómo conviven ambos mundos:

```
services:
  db:
    image: postgres:15
    # ... (configuracion previa)

  web_django: # El Monolito (v1)
    build: .
    command: gunicorn tienda_project.wsgi:application --bind
     0.0.0.0:8000
    depends_on:
      - db

  pagos_flask: # El Microservicio (v2) - EL ESTRANGULADOR
    build: ./microservicio_pagos
    ports:
      - "5000:5000" # Solo para pruebas iniciales, luego lo cerraremos

  nginx:
    image: nginx:1.25
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - web_django
      - pagos_flask
```

5. Paso 4: El Cerebro del Ruteo (nginx.conf)

Actualice su archivo `nginx/nginx.conf`. Aquí es donde sucede el estrangulamiento real. Nginx decidirá a quién enviar la petición según la versión de la API.

```
upstream django_v1 {
    server web_django:8000;
}

```

```
upstream flask_v2 {
    server pagos_flask:5000;
}

server {
    listen 80;

    # RUTA VIEJA: Sigue yendo a Django
    location /api/v1/ {
        proxy_pass http://django_v1;
        proxy_set_header Host $host;
    }

    # RUTA NUEVA: Estrangulamos la funcion de compra y la mandamos a Flask
    location /api/v2/comprar {
        proxy_pass http://flask_v2;
        proxy_set_header Host $host;
    }
}
```

Listing 3: nginx.conf actualizado

6. Paso 5: Despliegue en AWS

1. Suba los cambios a su repositorio. 2. En su instancia EC2, ejecute:

```
git pull origin main
docker compose up -d --build
```

7. Entregables y Evaluación

Para aprobar este laboratorio, el estudiante debe cargar un documento PDF con:

1. **Prueba de Coexistencia (v1):** Captura de pantalla de Postman haciendo un `GET` a `http://IP-AWS/api/v1/productos/`. Debe responder Django.
2. **Prueba de Estrangulamiento (v2):** Captura de pantalla de Postman haciendo un `POST` a `http://IP-AWS/api/v2/comprar`. El JSON de respuesta debe decir: *"procesada exitosamente por el Microservicio Flask"*.
3. **Logs de Orquestación:** Salida del comando `docker compose logs nginx` donde se vea el redireccionamiento de las peticiones a los dos backends distintos.